

Xây dựng mô hình phân loại chủ đề tin tức tiếng Việt

Trần Ngọc Hưng
Khoa Công nghệ Thông tin, Trường
Đại học Đà Lạt
Đà Lạt, Lâm Đồng, Việt Nam
2212377@dlu.edu.vn

Nguyễn Hoàng Nam Khánh
Khoa Công nghệ Thông tin, Trường
Đại học Đà Lạt
Đà Lạt, Lâm Đồng, Việt Nam
2212391@dlu.edu.vn

Trần Vũ Thành Luân
Khoa Công nghệ Thông tin, Trường
Đại học Đà Lạt
Đà Lạt, Lâm Đồng, Việt Nam
2212410@dlu.edu.vn

Tóm tắt nội dung

Phân loại chủ đề tin tức là một bài toán cốt lõi trong xử lý ngôn ngữ tự nhiên, đặc biệt cần thiết cho các hệ thống báo điện tử và tổng hợp tin tức tiếng Việt. Báo cáo này trình bày toàn bộ quy trình xây dựng một hệ thống phân loại tin tức tiếng Việt từ đầu đến cuối, gồm: (i) công cụ thu thập dữ liệu đa luồng từ 15 nguồn báo lớn, (ii) pipeline tiền xử lý kết hợp chuẩn hoá Unicode, lọc nhiễu và khử trùng tiêu đề bằng n-gram, (iii) mô hình phân loại tinh chỉnh trên *PhoBERT-base* với kỹ thuật cửa sổ trượt (sliding window, kích thước 256, bước nhảy 128) cho văn bản dài, kết hợp *R-Drop*, *label smoothing* và trọng số theo lớp. Ngoài ra, TF-IDF được dùng để phân tích đặc trưng và làm cơ sở so sánh. Trên tập dữ liệu sạch gồm 225.731 bài báo phân thành 11 chủ đề, mô hình đạt độ chính xác (Accuracy) 90,10% và F1 trọng số 0,9022 trên tập kiểm thử 45.147 mẫu. Một ứng dụng demo được triển khai trên *Streamlit* cho phép người dùng nhập văn bản và nhận về phân phối xác suất top-k cho 11 chủ đề.

Keywords

Phân loại văn bản; PhoBERT; Tin tức tiếng Việt; TF-IDF; Sliding window; Học sâu; NLP tiếng Việt

1 Giới thiệu

Sự bùng nổ của báo điện tử tiếng Việt trong những năm gần đây đã tạo ra một khối lượng tin tức khổng lồ được sản xuất hằng ngày. Để giúp độc giả tiếp cận thông tin theo đúng nhu cầu, các toà soạn cần một hệ thống **phân loại chủ đề** (topic classification) hoạt động chính xác và bền vững với sự đa dạng về văn phong, độ dài và cách dùng từ. Phân loại tin tức cũng là tiền đề cho hàng loạt tác vụ hạ nguồn như cá nhân hoá nội dung, gợi ý bài đọc, kiểm duyệt nội dung và phân tích xu hướng truyền thông.

Khác với tiếng Anh, tiếng Việt đặt ra nhiều thách thức cho các mô hình ngôn ngữ. Thứ nhất, tiếng Việt là ngôn ngữ *đơn lập* với từ ghép phổ biến, dẫn đến việc một “từ” về mặt ngữ nghĩa có thể được tạo từ hai âm tiết tách rời (ví dụ: “công nghệ”, “giáo dục”); việc tách từ không chuẩn xác sẽ làm sai lệch toàn bộ chuỗi token đầu vào của mô hình [7, 9]. Thứ hai, hệ thống dấu thanh và dấu phụ phong phú khiến cho các bộ tokenizer đa ngữ thường mất thông tin nếu không có chuẩn hoá Unicode đúng. Thứ ba, các bài báo Việt Nam thường rất dài (>1.000 token), vượt giới hạn cửa sổ ngữ cảnh của các kiến trúc Transformer chuẩn.

Trong báo cáo này, nhóm chúng tôi đóng góp ba nội dung chính: (1) một *công cụ thu thập dữ liệu* đa luồng dựa trên *newspaper3k* và *requests* cho 15 báo điện tử lớn của Việt Nam với cơ chế *checkpoint* chống mất dữ liệu khi gián đoạn; (2) một *pipeline xử lý dữ liệu* bao gồm chuẩn hoá Unicode NFC, làm sạch HTML, khử trùng tiêu đề bằng n-gram ký tự và lọc nhiễu; (3) một *mô hình phân loại* tinh chỉnh trên *PhoBERT-base* với kỹ thuật cửa sổ trượt cho văn

bản dài, đạt độ chính xác cao trên 11 chủ đề. Phần còn lại của bài báo được tổ chức như sau: Mục 2 trình bày các công trình liên quan; Mục 3 giới thiệu cơ sở lý thuyết; Mục 4 mô tả công cụ thu thập dữ liệu; Mục 5 trình bày tiền xử lý dữ liệu; Mục 6 liệt kê công nghệ sử dụng; Mục 7 trình bày quá trình huấn luyện và kết quả; Mục 8 mô tả ứng dụng demo; Mục 9 tổng kết và đề xuất hướng phát triển.

2 Công trình liên quan

Mô hình ngôn ngữ tiền huấn luyện. BERT [2] là kiến trúc mở đầu cho làn sóng *pre-trained language models* dựa trên Transformer, sử dụng mục tiêu *Masked Language Modeling* (MLM) và *Next Sentence Prediction*. RoBERTa [4] cải tiến BERT bằng cách bỏ NSP, huấn luyện lâu hơn trên dữ liệu lớn hơn và sử dụng *dynamic masking*, đạt kết quả vượt trội trên hàng loạt benchmark. Vaswani và cộng sự [14] đề xuất kiến trúc Transformer với cơ chế *self-attention*, là nền tảng cho cả BERT/RoBERTa.

PhoBERT cho tiếng Việt. Nguyen & Nguyen [8] phát triển PhoBERT, mô hình tiền huấn luyện riêng cho tiếng Việt theo paradigm RoBERTa trên tập 20GB văn bản đã tách từ. PhoBERT vượt qua các baseline đa ngữ như mBERT/XLM-R trên các tác vụ phân tích cảm xúc, gán nhãn từ loại và nhận dạng thực thể. Một ứng dụng tiêu biểu của PhoBERT là phân loại bệnh từ triệu chứng tiếng Việt.

TF-IDF và phân loại tin tức. Salton & Buckley [12] đề xuất các phương pháp gán trọng số từ trong truy hồi văn bản, trong đó TF-IDF đến nay vẫn là một baseline mạnh, đặc biệt khi dữ liệu hạn chế hoặc cần tốc độ inference cao. Phan & Nguyễn [3] áp dụng TF-IDF kết hợp các bộ phân loại cổ điển (Naive Bayes, SVM) cho tin tức tiếng Việt và chỉ ra rằng việc tách từ kỹ lưỡng giúp cải thiện đáng kể độ chính xác.

Văn bản dài và sliding window. Pappagari và cộng sự [11] đề xuất kiến trúc Transformer phân cấp cho văn bản dài: chia tài liệu thành các đoạn ngắn, mã hoá từng đoạn rồi tổng hợp ở tầng trên. Tư tưởng cửa sổ trượt (chia tài liệu thành các đoạn chồng lấp) là một cách hiện thực hoá đơn giản và hiệu quả của ý tưởng này, được chúng tôi áp dụng cho bài toán phân loại tin tức tiếng Việt.

Tách từ tiếng Việt. VnCoreNLP [9] và underthesea [1] là hai bộ công cụ tách từ chính xác, được sử dụng rộng rãi. Nguyen & Nguyen [10] đặt câu hỏi về tính cần thiết của tách từ cho phân loại cảm xúc tiếng Việt và chỉ ra rằng với mô hình BPE đủ tốt, hiệu quả của tách từ giảm đi đáng kể – một quan sát mà chúng tôi cũng tận dụng trong cấu hình thực nghiệm.

3 Cơ sở lý thuyết

3.1 Mô hình PhoBERT

3.1.1 Kiến trúc Transformer. Transformer [14] là kiến trúc mạng nơ-ron ra đời năm 2017, thay thế hoàn toàn các kết nối hồi

quy (RNN, LSTM) và tích chập (CNN) trong xử lý chuỗi bằng *cơ chế attention thuần*. Bản gốc Transformer có dạng encoder-decoder cho dịch máy: *encoder* mã hoá chuỗi nguồn thành biểu diễn ngữ cảnh, *decoder* sinh dần từng token đích. Tuy nhiên, với các tác vụ **phân loại văn bản** như bài toán của chúng tôi, chỉ cần phần *encoder*; do đó PhoBERT (và họ BERT/RoBERTa) chỉ giữ lại phần *encoder* – gọi là kiến trúc *encoder-only*.

Mỗi **lớp encoder** của Transformer gồm hai khối phụ (sub-layer) nối tiếp:

- (1) *Multi-head self-attention* – mỗi token đầu vào tự “nhìn” toàn bộ các token khác trong chuỗi để tổng hợp ngữ cảnh; cơ chế này được trình bày chi tiết ở Phương trình 6 và 7.
- (2) *Position-wise feed-forward network* (FFN) – một mạng neuron hai tầng áp dụng **độc lập theo từng vị trí** token:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (1)$$

trong đó: $x \in \mathbb{R}^{d_{\text{model}}}$ là vector biểu diễn của một vị trí token sau lớp attention; $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $b_1 \in \mathbb{R}^{d_{\text{ff}}}$ là tham số tầng ẩn (với PhoBERT-base $d_{\text{ff}} = 3072$); $\max(0, \cdot)$ là hàm kích hoạt ReLU; $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, b_2 là tham số chiếu về lại $d_{\text{model}} = 768$. FFN bổ sung khả năng biểu diễn phi tuyến mà attention (vốn chỉ là tổ hợp tuyến tính của value) thiếu.

Quanh mỗi sub-layer, Transformer dùng hai kỹ thuật ổn định huấn luyện:

- **Residual connection** (kết nối dư) cộng đầu vào trực tiếp vào đầu ra của sub-layer, giúp tín hiệu gradient lan truyền dễ dàng qua hàng chục lớp sâu mà không bị tiêu biến.
- **Layer Normalization** chuẩn hoá theo chiều đặc trưng:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \quad (2)$$

với x là input của sub-layer và $\text{Sublayer}(x)$ là đầu ra của sub-layer đó (attention hoặc FFN). Khác với *batch normalization*, LayerNorm chuẩn hoá trên từng mẫu nên hoạt động ổn định cả với batch size nhỏ.

Cuối cùng, do attention không có khái niệm thứ tự token, mỗi vị trí được cộng thêm một **positional encoding** vào embedding. Transformer gốc dùng sin/cosine:

$$\begin{aligned} PE_{(pos, 2i)} &= \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \\ PE_{(pos, 2i+1)} &= \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \end{aligned} \quad (3)$$

với pos là chỉ số vị trí (0, 1, 2, ...), i là chỉ số chiều embedding. PhoBERT (giống RoBERTa) sử dụng *learned positional embedding* (vector vị trí được học cùng các tham số khác), tối đa 258 vị trí cho phobert-base. Việc cộng PE vào embedding đầu vào cho phép mô hình phân biệt “*Cầu thủ ghi bàn*” với “*Bàn ghi cầu thủ*” mặc dù tập từ giống nhau.

Tóm lại, mỗi lớp encoder của PhoBERT có cấu trúc tuần tự: *Multi-head Self-Attention* → Residual + LayerNorm → FFN → Residual + LayerNorm. Kiến trúc này được lặp **12 lần** trong phobert-base (và 24 lần trong phobert-large), tạo nên một mạng sâu nhưng vẫn huấn luyện được ổn định nhờ residual và LayerNorm.

3.1.2 Lớp nhúng đầu vào và BPE tokenization. Trước khi đi vào ngăn xếp encoder, mỗi token được biểu diễn bởi **tổng ba vector nhúng**: token embedding $E_t \in \mathbb{R}^{|V| \times d_{\text{model}}}$ tra cứu trong từ điển BPE ($|V| = 64,001$ với PhoBERT), positional embedding $E_p \in$

$\mathbb{R}^{n_{\text{max}} \times d_{\text{model}}}$ (với $n_{\text{max}} = 258$) và segment/token-type embedding E_s (PhoBERT chỉ dùng một segment nên đóng góp gần như hằng số). Đầu vào của lớp encoder đầu tiên là:

$$x_0 = \text{LayerNorm}(E_t(t) + E_p(\text{pos}) + E_s) + \text{dropout} \quad (4)$$

Khác với BERT đa ngữ dùng WordPiece, PhoBERT sử dụng *Byte-Pair Encoding* (BPE) [13]: thuật toán xuất phát từ tập ký tự, lặp đi lặp lại việc *ghép cặp ký hiệu liền kề có tần suất đồng xuất hiện cao nhất* cho đến khi đạt kích thước từ vựng định trước. Với tiếng Việt, BPE đặc biệt phù hợp vì: (i) xử lý tốt từ ghép – một “đơn vị nghĩa” như “công nghệ” có thể được tách thành 1–2 subword ổn định; (ii) không bị vấn đề *out-of-vocabulary* do mọi từ lạ đều phân rã được về subword đã biết; (iii) giảm kích thước từ vựng đáng kể so với word-level (64.001 so với hàng trăm nghìn từ thuần tiếng Việt).

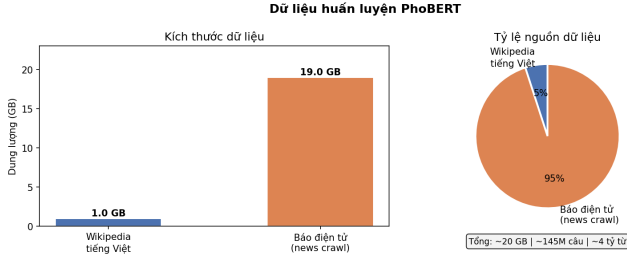
3.1.3 Mục tiêu tiền huấn luyện MLM. PhoBERT được tiền huấn luyện theo paradigm *Masked Language Modeling* (MLM) của RoBERTa [4]. Cho một chuỗi token đầu vào $x = (x_1, \dots, x_n)$, ta chọn ngẫu nhiên một tập chỉ số $\mathcal{M} \subset \{1, \dots, n\}$ với tỉ lệ 15% rồi thay các token tại các vị trí đó bằng token đặc biệt $\langle \text{mask} \rangle$ (80%), một token ngẫu nhiên trong từ vựng (10%) hoặc giữ nguyên (10%). Mục tiêu là khôi phục các token bị che dựa trên ngữ cảnh hai chiều:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \sum_{i \in \mathcal{M}} \log p_\theta(x_i | x_{\setminus \mathcal{M}}) \quad (5)$$

trong đó: $\mathcal{D}$ là phân phối kho dữ liệu tiền huấn luyện (20GB văn bản tiếng Việt); $x_{\setminus \mathcal{M}}$ là chuỗi đầu vào sau khi đã che các vị trí trong $\mathcal{M}$; p_θ là xác suất do mô hình (θ là toàn bộ tham số) gán cho từng token gốc tại vị trí bị che. Khác với BERT, PhoBERT dùng **dynamic masking** (sinh mặt nạ mới ở mỗi epoch thay vì cố định một lần) và **bỏ mục tiêu Next Sentence Prediction** – hai cải tiến trực tiếp kế thừa từ RoBERTa, giúp mô hình tận dụng dữ liệu hiệu quả hơn.

3.1.4 PhoBERT. PhoBERT [8] là mô hình ngôn ngữ tiền huấn luyện cho tiếng Việt, dựa trên kiến trúc **Transformer encoder** (không có decoder) và quy trình huấn luyện theo RoBERTa. Hai biến thể chính là phobert-base (12 lớp encoder, kích thước ẩn 768, 12 đầu attention, ~135M tham số) và phobert-large (24 lớp, ẩn 1024). Mô hình được tiền huấn luyện trên tập dữ liệu khoảng 20GB văn bản tiếng Việt đã tách từ với mục tiêu *Masked Language Modeling* (xem Mục 3.1.3). Một đặc điểm quan trọng là PhoBERT **khuyến nghị tách từ trước khi tokenization** bằng VnCoreNLP, do tokenizer của nó được huấn luyện trên dữ liệu đã tách từ ghép bằng dấu gạch dưới “_”. Lý do chỉ giữ phần encoder cho phân loại văn bản: tác vụ không yêu cầu sinh chuỗi đầu ra nên không cần khối decoder (vốn có thêm *masked self-attention* để đảm bảo tự hồi quy và *cross-attention* giữa decoder–encoder); encoder thuần tuý vừa rẻ vừa cung cấp biểu diễn ngữ cảnh hai chiều cho mọi vị trí token, phù hợp với mọi tác vụ phân loại/gán nhãn.

Hình 1 cho thấy gần **95%** kho tiền huấn luyện của PhoBERT đến từ *báo điện tử tiếng Việt*, chỉ một phần nhỏ đến từ Wikipedia. Tỉ trọng áp đảo của miền tin tức trong dữ liệu tiền huấn luyện là một trong những lý do chính khiến PhoBERT đặc biệt phù hợp với bài toán phân loại tin tức của chúng tôi: phân phối từ vựng, văn phong báo chí và cấu trúc câu mà mô hình đã “thấy” rất gần với phân phối thực tế của dữ liệu hạ nguồn.



Hình 1: Cơ cấu tập tiền huấn luyện 20GB của PhoBERT: ~1GB Wikipedia tiếng Việt cộng với ~19GB báo điện tử (news crawl), tổng ~145 triệu câu và ~4 tỷ token.

Trái tim của Transformer là cơ chế *scaled dot-product attention* [14]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (6)$$

trong đó các ký hiệu được giải thích như sau:

- $Q \in \mathbb{R}^{n \times d_k}$: ma trận *query*, mỗi hàng là vector “câu hỏi” của một token, biểu diễn nội dung mà token đó muốn “hỏi” các token khác.
- $K \in \mathbb{R}^{n \times d_k}$: ma trận *key*, mỗi hàng là vector “khóa” của một token, biểu diễn nội dung mà token đó cung cấp khi được truy vấn.
- $V \in \mathbb{R}^{n \times d_v}$: ma trận *value*, chứa thông tin thực sự được tổng hợp lại sau khi đã tính trọng số chú ý.
- n : độ dài chuỗi đầu vào (số token sau khi tokenize, ở cấu hình của chúng tôi $n \leq 256$).
- d_k : số chiều của không gian *query/key* (với PhoBERT-base, $d_k = 768/12 = 64$ cho mỗi head).
- $QK^T \in \mathbb{R}^{n \times n}$: ma trận điểm tương đồng giữa mọi cặp token; phần tử (i, j) đo mức độ liên quan của token j đối với token i .
- $\sqrt{d_k}$: hệ số chuẩn hoá; khi d_k lớn, tích vô hướng có phương sai cao khiến softmax quá “cứng”, phép chia này giữ gradient ổn định.
- $\text{softmax}(\cdot)$: chuẩn hoá theo hàng để mỗi token nhận một phân phối xác suất chú ý lên các token khác (tổng bằng 1).

Cách hoạt động của scaled dot-product attention. Để hiểu rõ cơ chế này, hãy xem xét một ví dụ cụ thể với câu tiếng Việt: “*Câu thủ ghi bàn thắng*”. Khi xử lý từ “ghi”, mô hình cần xác định xem từ này liên quan đến từ nào khác trong câu. Cơ chế attention hoạt động qua ba bước:

Bước 1 – Tính điểm tương đồng: Với mỗi token i (ví dụ “ghi”), vector query q_i của nó được nhân với tất cả các vector key k_j của các token khác để tính điểm tương đồng $s_{ij} = q_i \cdot k_j$. Điểm này đo mức độ “liên quan” giữa token i và token j . Trong ví dụ, “ghi” sẽ có điểm cao với “cầu thủ” (chủ ngữ) và “bàn thắng” (tân ngữ).

Bước 2 – Chuẩn hoá và softmax: Ma trận điểm QK^T được chia cho $\sqrt{d_k}$ để ổn định gradient. Khi d_k lớn (ví dụ 64), tích vô hướng có phương sai cao, khiến một vài giá trị rất lớn và phần còn lại rất nhỏ; sau khi qua softmax, gradient sẽ gần như triệt tiêu ở hầu hết vị trí. Phép chia $\sqrt{d_k}$ đưa phương sai về 1, giữ softmax ở vùng có gradient tốt. Sau đó, softmax biến mỗi hàng thành phân phối xác

suất: $\alpha_{ij} = \frac{\exp(s_{ij}/\sqrt{d_k})}{\sum_k \exp(s_{ik}/\sqrt{d_k})}$, trong đó α_{ij} là “trọng số chú ý” mà token i dành cho token j .

Bước 3 – Tổng hợp thông tin: Biểu diễn đầu ra của token i là tổ hợp tuyến tính của các vector value: $\text{output}_i = \sum_j \alpha_{ij} v_j$. Nếu “ghi” chú ý mạnh đến “cầu thủ” ($\alpha = 0,6$) và “bàn thắng” ($\alpha = 0,3$), thì biểu diễn mới của “ghi” sẽ mang 60% thông tin từ “cầu thủ” và 30% từ “bàn thắng”, giúp mô hình hiểu đúng vai trò động từ của “ghi” trong ngữ cảnh thể thao.

Cơ chế *multi-head attention* cho phép mô hình quan sát nhiều “khía cạnh” phụ thuộc cùng lúc:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (7)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

trong đó: h là số đầu chú ý (với PhoBERT-base $h = 12$); $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ là các ma trận chiếu tuyến tính riêng cho head i , mỗi head học một “khía cạnh” phụ thuộc khác nhau (ví dụ phụ thuộc cú pháp, phụ thuộc ngữ nghĩa xa, đồng tham chiếu); $\text{head}_i \in \mathbb{R}^{n \times d_v}$ là kết quả attention của head i ; Concat ghép h ma trận này theo chiều cột; và $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ là ma trận chiếu đầu ra hợp nhất các head về lại không gian biểu diễn ban đầu.

Để dùng PhoBERT cho phân loại văn bản, chúng tôi lấy embedding của token đặc biệt [CLS] (ở vị trí đầu chuỗi) làm biểu diễn của toàn bộ câu, sau đó đưa qua một tầng *linear classifier* và *softmax*:

$$p(y = c | x) = \text{softmax}(W_c \cdot h_{[\text{CLS}]} + b_c) \quad (8)$$

trong đó: x là chuỗi token đầu vào; y là biến nhãn dự đoán; $c \in \{1, \dots, C\}$ là chỉ số lớp (với $C = 11$ chủ đề); $h_{[\text{CLS}]} \in \mathbb{R}^{768}$ là vector embedding của token đặc biệt [CLS] sau khi qua toàn bộ 12 lớp encoder, đóng vai trò “vector tóm tắt” cả chuỗi; $W_c \in \mathbb{R}^{1 \times 768}$ và $b_c \in \mathbb{R}$ là trọng số và độ lệch của tầng phân loại tuyến tính cho lớp c . Toàn bộ tham số W_c, b_c và toàn bộ trọng số PhoBERT đều được *cùng tinh chỉnh* (fine-tune) trên dữ liệu phân loại tin tức.

3.2 TF-IDF

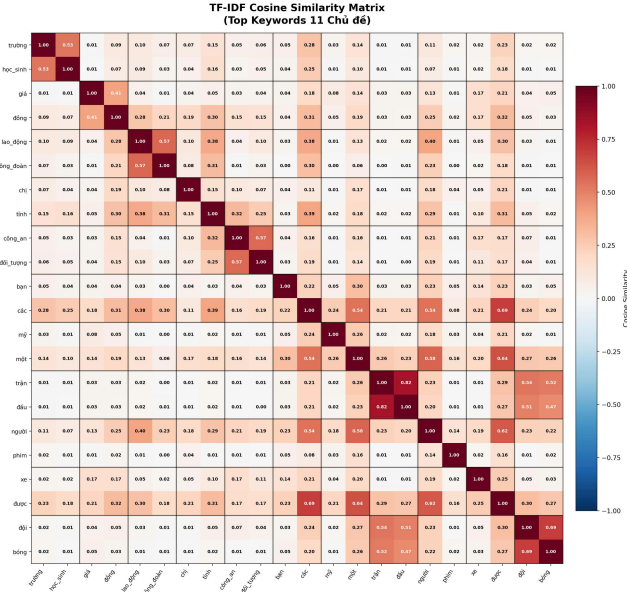
TF-IDF (Term Frequency – Inverse Document Frequency) [12] là một phương pháp gán trọng số từ kinh điển trong truy hồi và phân loại văn bản:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log \frac{N}{1 + \text{DF}(t)} \quad (9)$$

trong đó: t là một từ (term) trong từ điển; d là một tài liệu cụ thể; $\text{TF}(t, d)$ là số lần từ t xuất hiện trong tài liệu d (có thể chuẩn hoá theo độ dài tài liệu); N là tổng số tài liệu trong toàn bộ kho ngữ liệu; $\text{DF}(t)$ là số tài liệu trong kho có chứa ít nhất một lần từ t ; hệ số 1 ở mẫu số là kỹ thuật *smoothing* để tránh chia cho 0 khi $\text{DF}(t) = 0$. Trực giác đằng sau công thức là: một từ có giá trị phân biệt cao nếu nó xuất hiện thường xuyên trong một tài liệu cụ thể (TF cao) nhưng hiếm gặp trên toàn kho (DF thấp, nên log-IDF cao); ngược lại, các *stopword* như “và”, “là” có DF rất lớn nên trọng số TF-IDF gần như bằng 0.

Trong dự án, TF-IDF được dùng cho ba mục tiêu: (1) phân tích đặc trưng – liệt kê các từ khoá đại diện cho từng chủ đề, (2) làm vector đầu vào cho baseline (*Logistic Regression/SVM*) để so sánh với PhoBERT, và (3) phục vụ tác vụ khử trùng tiêu đề (xem Mục 3.4).

Trong thực hành, chúng tôi sử dụng cài đặt `TfidfVectorizer` của `scikit-learn`, vốn áp dụng hai cải tiến nhỏ so với công thức gốc: *sublinear TF* thay $TF(t, d)$ bằng $1 + \log TF(t, d)$ để giảm ảnh hưởng của các từ lặp nhiều lần trong một bài, và *chuẩn hoá L2* mỗi hàng để $\|v_d\|_2 = 1$. Việc chuẩn hoá L2 đặc biệt thuận lợi cho công thức cosine ở Mục 3.4: khi cả hai vector đã chuẩn hoá, độ tương đồng cosine rút gọn về tích vô hướng và phép tính trở nên rất nhanh trên ma trận thưa. Hình 2 minh hoạ một ma trận TF-IDF của các bài báo trong dữ liệu thu thập.



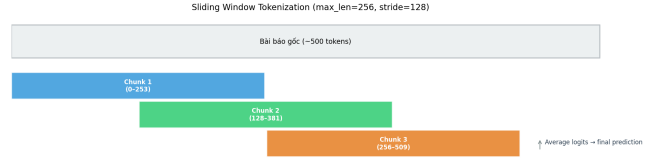
Hình 2: Một phần ma trận TF-IDF được trích từ tập tin tức tiếng Việt thu thập được; mỗi hàng là một bài báo, mỗi cột là một từ, độ đậm của ô tương ứng với giá trị TF-IDF của từ trong bài.

Quan sát Hình 2, có thể thấy ma trận rất *thưa* (sparse): phần lớn ô có giá trị gần 0 vì mỗi bài chỉ chứa một tỉ lệ nhỏ trên tổng số từ vựng. Các ô đậm tập trung theo cột vào những từ chuyên biệt cho từng chủ đề (ví dụ “cầu thủ”, “bản thắng” đậm ở các bài thể thao; “cổ phiếu”, “thị trường” đậm ở bài kinh doanh). Tính thưa và tính phân cụm theo chủ đề chính là cơ sở để các bộ phân loại tuyến tính trên TF-IDF hoạt động hiệu quả.

3.3 Kỹ thuật cửa sổ trượt cho văn bản dài

PhoBERT, giống như nhiều mô hình kiểu BERT, có giới hạn cứng về độ dài chuỗi đầu vào (256 token trong cấu hình của chúng tôi, tối đa 512). Tuy nhiên, một bài báo tiếng Việt thường dài 800–1.500 token sau khi BPE-tokenize. Cắt cụt (truncation) sẽ làm mất thông tin từ phần thân bài, vốn chứa nhiều từ khoá quan trọng cho việc phân loại chủ đề [11].

Chúng tôi áp dụng *cửa sổ trượt* (sliding window): mỗi tài liệu D được chia thành các cửa sổ chồng lấp $W_1, W_2, \dots, W_K$ với kích thước cửa sổ $L = 256$ và bước trượt $s = 128$ (chồng lấp 50%). Mỗi cửa sổ được mã hoá độc lập bởi PhoBERT, sau đó các vector logits được *trung bình hoá* (mean pooling) để cho ra phân phối xác suất duy nhất cho toàn tài liệu. Hình 3 minh hoạ cách cắt cửa sổ.



Hình 3: Minh hoạ kỹ thuật cửa sổ trượt: tài liệu dài được chia thành các cửa sổ 256 token chồng lấp 128 token; mỗi cửa sổ được phân loại độc lập, các logits được trung bình hoá để cho ra phân phối xác suất duy nhất.

Có thể thấy rõ hai ưu điểm của sơ đồ ở Hình 3: (1) phần *chồng lấp* 128 token giữa các cửa sổ kế tiếp đảm bảo các cụm từ nằm vắt qua biên không bị mất ngữ cảnh; (2) việc trung bình hoá logits có tác dụng giảm phương sai dự đoán, đặc biệt khi một số cửa sổ chứa đoạn “trung tính” (lời giới thiệu, đoạn dẫn) ít mang dấu hiệu chủ đề.

Số cửa sổ sinh ra trên một tài liệu có n token là $K = \lceil \max(n - L, 0) / s \rceil + 1$. Với phân phối độ dài $n \in [300, 1500]$ token đo trên kho dữ liệu của chúng tôi, mỗi bài sinh trung bình 5–10 cửa sổ và do đó chi phí inference tăng tuyến tính theo độ dài bài thay vì bị cắt cụt. Về lựa chọn chiến lược tổng hợp, chúng tôi đã so sánh ba phương án: *mean logits* (trung bình logits chưa softmax), *max logits* (lấy max theo từng lớp) và *majority vote* (mỗi cửa sổ một phiếu). Mean logits được chọn cho cấu hình cuối cùng vì: (i) giảm phương sai dự đoán tốt nhất khi nhiều cửa sổ “trung tính”; (ii) giữ thông tin về độ tin cậy thay vì rời rạc hoá; (iii) khả vi – cho phép áp dụng ngay cả ở pha huấn luyện theo *chunk-level* mà không cần một bước hậu xử lý không khả vi. Mã giả (pseudo-code) của thủ tục dự đoán với cửa sổ trượt được trình bày trong Thuật toán 1; Listing 1 là phần lõi cài đặt thực tế trong mã nguồn dự án.

Algorithm 1 Dự đoán với cửa sổ trượt

Require: Tài liệu D , mô hình f_θ , $L = 256$, $s = 128$

Ensure: Phân phối xác suất $p \in \mathbb{R}^C$

```

1:  $T \leftarrow \text{tokenize}(D)$ 
2:  $\mathcal{W} \leftarrow \emptyset$ 
3: for  $i = 0, s, 2s, \dots$  while  $i < |T|$  do
4:    $W_i \leftarrow T[i : i + L]$ 
5:    $\mathcal{W} \leftarrow \mathcal{W} \cup \{W_i\}$ 
6: end for
7:  $\mathbf{z} \leftarrow \frac{1}{|\mathcal{W}|} \sum_{W \in \mathcal{W}} f_\theta(W)$ 
8:  $p \leftarrow \text{softmax}(\mathbf{z})$ 
9: return  $p$ 

```

3.4 Phân cụm ký tự trùng nhau trên tiêu đề

Khi gộp dữ liệu từ 15 nguồn khác nhau, các bài báo được dẫn lại thường xuất hiện trùng tiêu đề hoặc chỉ khác một vài ký tự (do thay đổi dấu câu, viết hoa). Các bản sao gần đúng (*near-duplicates*) làm sai lệch phân phối nhãn và gây rò rỉ giữa tập huấn luyện và tập kiểm thử nếu không xử lý.

Chúng tôi áp dụng quy trình khử trùng dựa trên *n-gram ký tự*: (1) chuẩn hoá tiêu đề (Unicode NFC, lowercase, loại bỏ khoảng trắng thừa); (2) trích các *shingle* 3-gram ký tự cho mỗi tiêu đề; (3) xây

Listing 1: Lỗi sinh của số trượt cho một bài (rút gọn từ `text_utils.py`).

```

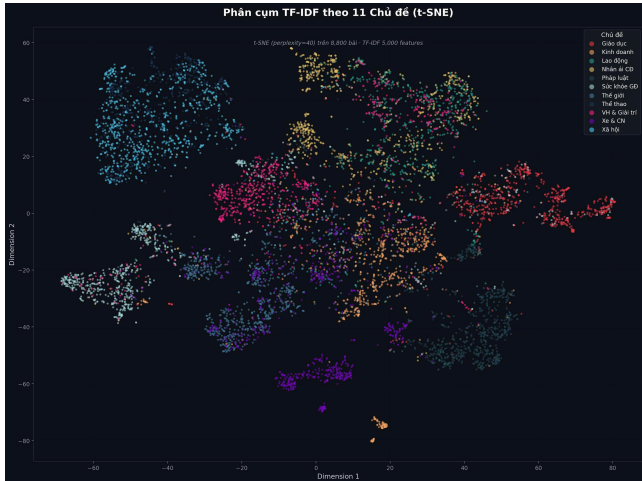
1 ids = tokenizer(text, add_special_tokens=False)["input_ids"]
2 usable = max_length - 2 # tru <s>, </s>
3 start, chunks = 0, []
4 while start < len(ids):
5     end = min(start + usable, len(ids))
6     win = [bos] + ids[start:end] + [eos]
7     win += [pad] * (max_length - len(win))
8     chunks.append(win)
9     if end >= len(ids):
10         break
11     start += stride

```

dựng vector TF-IDF cấp ký tự; (4) tính độ tương đồng cosine giữa tất cả các cặp tiêu đề; (5) gom cụm các cặp có độ tương đồng $\geq 0,85$ bằng *Union-Find*; (6) trong mỗi cụm chỉ giữ lại bài có nội dung dài nhất. Hình 4 minh họa kết quả phân cụm các tiêu đề trùng. Cosine giữa hai tiêu đề được tính theo công thức:

$$\text{sim}(t_i, t_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\| \cdot \|\mathbf{v}_j\|} \quad (10)$$

trong đó: t_i, t_j là hai tiêu đề bất kỳ trong kho dữ liệu; $\mathbf{v}_i, \mathbf{v}_j \in \mathbb{R}^{|V|}$ là vector TF-IDF được tính ở cấp *n-gram ký tự* (mỗi chiều ứng với một shingle 3-gram trong từ vựng V); $\mathbf{v}_i \cdot \mathbf{v}_j$ là tích vô hướng; $\|\cdot\|$ là chuẩn Euclide; giá trị $\text{sim}(t_i, t_j) \in [0, 1]$, càng gần 1 hai tiêu đề càng giống nhau về thành phần ký tự. Ngưỡng 0,85 được chọn dựa trên khảo sát thủ công: dưới ngưỡng này, đa số cặp là tiêu đề khác chủ đề chỉ chia sẻ một vài cụm từ; trên ngưỡng này, hầu như là cùng một tin được nhiều báo đăng lại.



Hình 4: Kết quả phân cụm các tiêu đề trùng nhau theo *n-gram* ký tự và cosine: mỗi cụm (đường viền) là một tin tức được nhiều báo dẫn lại với cách viết tiêu đề hơi khác.

Hình 4 cho thấy các cụm tiêu đề trùng có kích thước đa dạng: cụm nhỏ chỉ 2–3 tiêu đề (thường là cùng một bài AP/Reuters được hai báo Việt dịch lại), cụm lớn lên đến hơn 10 tiêu đề (sự kiện thời sự lớn được đa số báo đưa tin). Sau khi giữ lại bản dài nhất trong mỗi cụm, hơn 12.000 bài trùng đã được loại khỏi tập dữ liệu. Qua đó tránh rò rỉ thông tin từ tập huấn luyện sang tập kiểm thử. Về độ phức tạp, việc tính cosine giữa mọi cặp tiêu đề có chi phí $O(N^2|V|)$

trên ma trận thưa; với $N \approx 2,3 \times 10^5$ tiêu đề, chúng tôi cài đặt theo lô (batch) và lọc sơ bộ bằng cây tiền tố trên 3-gram đầu tiên để cắt giảm số cặp ứng viên hơn hai bậc, đưa chi phí thực tế xuống vài phút trên một CPU đa luồng. Trong các kho dữ liệu lớn hơn ($N \geq 10^6$), có thể thay bằng *MinHash + Locality-Sensitive Hashing* để đạt độ phức tạp tuyến tính kỳ vọng.

3.5 Các kỹ thuật ổn định huấn luyện

Ngoài hàm mất mát cơ bản *cross-entropy*, mô hình của chúng tôi sử dụng đồng thời bốn cơ chế ổn định và chống quá khớp: *R-Drop*, *label smoothing*, *AdamW + cosine schedule*, và *class weights*.

R-Drop [15] buộc hai lượt *forward* cùng một mẫu (với hai mặt nạ dropout độc lập) cho ra phân phối xác suất tương tự nhau. Cụ thể, đặt z_1, z_2 là logits của hai lượt forward, $p_1 = \text{softmax}(z_1)$, $p_2 = \text{softmax}(z_2)$; hàm mất mát R-Drop là:

$$\mathcal{L} = \frac{1}{2}(\mathcal{L}_{\text{CE}}(z_1, y) + \mathcal{L}_{\text{CE}}(z_2, y)) + \frac{\alpha}{2}(D_{\text{KL}}(p_1 \| p_2) + D_{\text{KL}}(p_2 \| p_1)) \quad (11)$$

trong đó y là nhãn thật, α là hệ số cân bằng giữa thành phần phân loại và thành phần ràng buộc nhất quán; D_{KL} là độ phân kỳ Kullback–Leibler đối xứng. R-Drop có thể được xem như một dạng *regularization* đặc biệt tinh vi: thay vì giảm trực tiếp dropout, nó dễ mặc cho dropout “làm việc” nhưng phạt mô hình nếu hai cấu hình dropout khác nhau dẫn đến hai dự đoán quá khác biệt. Trong báo cáo này, chúng tôi dùng $\alpha = 2,0$ – giá trị được khuyến nghị trong bài gốc cho các tác vụ phân loại. Listing 2 trình bày lỗi cài đặt R-Drop trong dự án.

Listing 2: Lỗi hàm mất mát R-Drop (rút gọn từ `train_v4.py`).

```

1 def sym_kl(z1, z2):
2     p = F.softmax(z1, -1)
3     q = F.softmax(z2, -1)
4     kl1 = F.kl_div(F.log_softmax(z1, -1), q,
5                   reduction="batchmean")
6     kl2 = F.kl_div(F.log_softmax(z2, -1), p,
7                   reduction="batchmean")
8     return 0.5 * (kl1 + kl2)
9
10 z1 = model(**inputs).logits
11 z2 = model(**inputs).logits # 2 mat na dropout khác nhau
12 loss = 0.5 * (ce(z1, y) + ce(z2, y)) + alpha * sym_kl(z1, z2)

```

Label smoothing [6] thay đích phân loại từ phân phối *one-hot* bằng một phân phối hơi mềm:

$$\tilde{y}_c = (1 - \varepsilon) y_c + \frac{\varepsilon}{C} \quad (12)$$

trong đó $\varepsilon \in [0, 1]$ là cường độ làm phẳng, $y_c \in \{0, 1\}$ là phân phối one-hot gốc và C là số lớp. Với $\varepsilon = 0,05$ (như trong cấu hình của chúng tôi), lớp đúng có xác suất đích 0,955 còn 10 lớp còn lại mỗi lớp 0,005. Lợi ích thực nghiệm: (i) tránh hiện tượng quá tự tin – đặc biệt quan trọng cho các chủ đề ngữ nghĩa chồng lấp (*lao_dong*, *xa_hoi*); (ii) cải thiện hiệu chuẩn (*calibration*), cho phép ngưỡng tin cậy được sử dụng đáng tin cậy ở pha triển khai.

AdamW và cosine schedule [5]. AdamW tách thành phần *weight decay* ra khỏi gradient của AdamW gốc, cho ra cập nhật tham số:

$$\theta_t \leftarrow \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda \theta_t \right) \quad (13)$$

với $\hat{m}_t, \hat{v}_t$ là moment thứ nhất/thứ hai đã hiệu chỉnh bias, λ là hệ số weight decay (0,01 trong cấu hình của chúng tôi). Tốc độ học η_t tuân theo lịch *linear warmup* trong 6% bước đầu rồi giảm theo *cosine decay*:

$$\eta_t = \frac{1}{2} \eta_0 \left(1 + \cos \frac{\pi(t - T_w)}{T - T_w} \right), \quad t \geq T_w \quad (14)$$

với $\eta_0 = 2 \times 10^{-5}$, T là tổng số bước huấn luyện và T_w là số bước warmup. Cấu hình này giúp các epoch sau “tĩnh chỉnh nhẹ nhàng” thay vì xoá kiến thức đã học – một đặc tính rất phù hợp cho fine-tuning các mô hình tiền huấn luyện lớn.

Class weights. Như đã đề cập ở Mục 5, chúng tôi dùng trọng số “balanced” của *scikit-learn*: $w_c = N/(C \cdot n_c)$ (Phương trình 15). Trọng số này được truyền trực tiếp vào tham số *weight* của *CrossEntropyLoss*, tác động tới cả thành phần CE trong R-Drop (Phương trình 11).

4 Công cụ thu thập dữ liệu

4.1 Mục tiêu thiết kế

Để có một kho dữ liệu đủ lớn, đa dạng và phản ánh ngữ cảnh báo chí Việt Nam đương thời, nhóm chúng tôi đã tự phát triển một bộ công cụ thu thập dữ liệu chuyên dụng. Bộ công cụ được thiết kế nhằm đáp ứng bốn yêu cầu chính: (i) *đa nguồn* – có khả năng cào song song nhiều báo với cấu trúc HTML khác nhau; (ii) *quy mô lớn* – thu được hàng chục nghìn bài trong một phiên chạy; (iii) *bền vững* – chống đứt mạng, chống chặn IP và có khả năng khôi phục sau gián đoạn; (iv) *sạch* – trích xuất chính xác phần thân bài, loại bỏ quảng cáo, chú thích ảnh, bình luận và nội dung dẫn liên kết.

4.2 Danh sách nguồn dữ liệu

Bộ công cụ hỗ trợ tổng cộng **15 báo điện tử lớn** trải đều trên các nhóm chuyên đề và các nhóm chủ quản: nhóm báo điện tử thương mại, nhóm báo chính trị – xã hội, nhóm chuyên ngành kinh tế và nhóm thông tấn. Việc đa dạng hoá nguồn giúp giảm thiên lệch về văn phong và phong cách đặt tiêu đề – điều rất quan trọng để mô hình phân loại không bị “học vẹt” vào một kiểu hành văn riêng của một toà soạn. Bảng 1 liệt kê đầy đủ 15 nguồn cùng tên miền.

Bảng 1: Danh sách 15 nguồn báo điện tử được công cụ hỗ trợ (rút từ tệp cấu hình *Tool/config.py*).

Nhóm	Báo	Tên miền
Thương mại	VnExpress	vnexpress.net
	Tuổi Trẻ	tuoitre.vn
	Dân Trí	dantri.com.vn
	Thanh Niên	thanhnien.vn
	VietnamNet	vietnamnet.vn
Chính trị – xã hội	Nhân Dân	nhandan.vn
	Báo Chính phủ	baochinhphu.vn
	Quân đội ND	qndd.vn
	CAND Online	cand.com.vn
Kinh tế	VnEconomy	vneconomy.vn
	Báo Đầu tư	baodautu.vn
	Lao Động	laodong.vn
Thông tấn	VietnamPlus	vietnamplus.vn
	VTV News	vtv.vn
	(nguồn phụ)	–

4.3 Phương pháp thu thập

Quy trình cào của một nguồn đi qua hai pha. *Pha thứ nhất – liệt kê URL*: bộ công cụ duyệt qua trang chuyên mục (mục lớn như “Kinh doanh”, “Thể thao...”) và các trang phân trang con để gom toàn bộ liên kết bài viết theo từng chuyên mục. Cụ thể, công cụ sử dụng thư viện *requests* để tải trang HTML của từng chuyên mục, sau đó dùng *BeautifulSoup4* với parser *lxml* để phân tích cấu trúc DOM và trích xuất các thẻ `<a>` chứa liên kết bài viết. Mỗi nguồn báo có cấu trúc HTML riêng, do đó công cụ duy trì một bộ *selector CSS* đặc thù cho từng tên miền (ví dụ với VnExpress: `article.item-news > h3 > a`, với Tuổi Trẻ: `div.box-category-item > h3 > a`). Để xử lý phân trang, công cụ phát hiện tự động nút “Trang tiếp” hoặc tăng dần tham số `?page=N` trong URL cho đến khi không còn bài mới.

Pha thứ hai – bóc tách nội dung: với mỗi URL, công cụ tải HTML đầy đủ, sau đó áp dụng một quy trình hai bước. Bước thứ nhất dùng thư viện *newspaper3k* – một công cụ trích xuất tự động dựa trên các heuristics phân tích cấu trúc DOM (ví dụ phát hiện thẻ `<article>`, mật độ văn bản cao, loại bỏ sidebar/footer) – để lấy nhanh tiêu đề, mô tả ngắn, ngày đăng, tác giả và phần thân bài. Bước thứ hai chạy bộ chọn lọc CSS riêng cho mỗi nguồn để xác minh và bổ sung những trường có cấu trúc đặc thù (ví dụ “chuyên mục con”, “thẻ bài”, “mục lục liên quan”). Hai bước này bổ sung lẫn nhau: *newspaper3k* cho tốc độ và mức bao phủ (hoạt động tốt trên 70–80% bài), còn bộ chọn lọc thủ công cho độ chính xác cao trên các trường mà heuristics dễ bỏ sót.

Ví dụ, với nguồn VnExpress, quy trình cào một bài cụ thể diễn ra như sau: (1) tải HTML từ URL `https://vnexpress.net/the-thao/bong-da/...`; (2) *newspaper3k* trích xuất tiêu đề từ thẻ `<h1 class="title-detail">`, nội dung từ `<article class="fck_detail">`, ngày đăng từ `<span class="date">`; (3) bộ chọn lọc CSS bổ sung trích xuất chuyên mục từ `breadcrumb ul.breadcrumb > li > a` và danh sách thẻ từ `div.tags > a`; (4) kết quả được gộp thành một bản ghi JSON chứa bảy trường: *url*, *title*, *description*, *content*, *published_date*, *author*, *category*, *tags*.

4.4 Cơ chế bền vững và chống chặn

Để hạn chế bị nguồn chặn IP, công cụ áp dụng nhiều kỹ thuật cùng lúc: xoay vòng *User-Agent* ngẫu nhiên trong một danh sách trình duyệt phổ biến; đặt độ trễ ngẫu nhiên giữa các yêu cầu để mô phỏng hành vi người dùng; tự động chuyển sang phiên bản di động của trang khi phiên bản máy tính trả về mã lỗi 4xx/5xx; và áp dụng cơ chế *thử lại theo cấp số nhân* (exponential backoff) khi gặp lỗi mạng tạm thời. Độ trễ được hiệu chỉnh *theo từng tên miền*: các báo có cơ chế chống bot mạnh hơn như Quân đội Nhân dân và Công an Nhân dân được đặt độ trễ 1,8–3,0 s và 1,2–2,2 s giữa hai yêu cầu, trong khi các báo còn lại chỉ cần 0,12–0,35 s.

Một thành phần quan trọng là **cơ chế checkpoint**: trong suốt quá trình cào, công cụ ghi định kỳ trạng thái xử lý (chuyên mục đang chạy, danh sách URL đã hoàn thành và URL còn lại) ra một tệp tiến độ. Khi tiến trình bị gián đoạn – vì lý do người dùng dừng, mất mạng hay máy treo – ở lần chạy kế tiếp công cụ sẽ đọc lại tệp tiến độ và tiếp tục chính xác từ điểm dừng, không cào lại các URL đã hoàn

thành. Cơ chế này đặc biệt hữu ích trong các phiên cào kéo dài hàng chục giờ.

4.5 Song song hoá và tổ chức đầu ra

Để rút ngắn thời gian cào, các nguồn được phân phối cho một *pool* luồng (thread pool) chạy song song; mỗi nguồn chiếm một luồng và bên trong luồng đó các yêu cầu được tuần tự để tôn trọng độ trễ tối thiểu giữa các lần truy cập cùng một máy chủ. Cách tổ chức này tận dụng tốt thời gian chờ I/O mà không gây áp lực bất thường lên một nguồn đơn lẻ.

Sau khi mỗi bài được trích xuất, kết quả được lưu thành một bản ghi gồm bảy trường: liên kết gốc, tiêu đề, mô tả tóm tắt, nội dung thân bài, ngày đăng, tác giả, chuyên mục/chủ đề và danh sách thẻ. Các bản ghi được nối thêm vào một tệp dữ liệu dạng bảng (CSV) và song song một tệp dạng dòng JSON để tiện cho cả phân tích thống kê lẫn nạp vào pipeline học máy.

Trùng nội dung giữa các nguồn được phát hiện ngay ở pha cào bằng *khoá MD5 trên 100 từ đầu* của thân bài; Listing 3 cho thấy lỗi cài đặt. Một tập băm dùng chung (`global_hashes`) được truy cập có khoá giữa các luồng để đảm bảo nguyên tử khi nhiều nguồn cào song song. Cơ chế băm tuy thô (chỉ chống các bản sao gần đúng) nhưng đủ để giảm đáng kể trùng lặp *trong quá trình cào*, trước khi pipeline khử trùng tiêu đề ở Mục 3.4 xử lý nốt các trường hợp tinh vi hơn.

Listing 3: Khử trùng theo MD5 của 100 từ đầu (rút gọn từ Tool/crawlers/crawl_all.py).

```
1 def dedup(data):
2     seen, out = set(), []
3     for item in data:
4         text = item.get('noi_dung', '')
5         words = text.lower().split()[:100]
6         key = hashlib.md5(''.join(words).encode()).hexdigest()
7         if key not in seen:
8             seen.add(key)
9             out.append(item)
10    return out
```

4.6 Quy mô dữ liệu

Bộ dữ liệu do nhóm thu thập được gồm khoảng **15.000 bài báo** mới từ 15 nguồn nói trên. Để mở rộng quy mô và đảm bảo phân bố nhãn đa dạng, chúng tôi gộp thêm một bộ dữ liệu tin tức tiếng Việt công khai chứa khoảng 300.000 bài. Sau bước làm sạch và khử trùng (Mục 5), kho dữ liệu cuối cùng còn **225.731 bài** được phân vào 11 nhãn chủ đề. Bảng 2 thống kê chi tiết phân bố này. Có thể thấy phân bố không đồng đều: lớp `the_thao` đông gấp ~16 lần lớp `xa_hoi`, là một thách thức trực tiếp cho pha huấn luyện và cũng là động lực của các kỹ thuật cân bằng được trình bày ở Mục 7.

Bảng 2: Phân bố dữ liệu theo 11 chủ đề trong tập huấn luyện và kiểm thử (sau khử trùng và làm sạch).

Chủ đề	Số bài (huấn luyện)	Số bài (kiểm thử)
giao_duc	21.834	5.459
kinh_doanh	19.476	4.870
lao_dong	3.476	870
nhan_ai_cong_dong	11.346	2.836
phap_luat	26.514	6.629
suc_khoe_gia_dinh	14.864	3.716
the_gioi	22.760	5.690
the_thao	28.751	7.187
van_hoa_giai_tri	16.450	4.112
xe_cong_nghe	13.350	3.338
xa_hoi	1.763	440
Tổng	180.584	45.147

5 Xử lý dữ liệu

Pipeline tiền xử lý được tổ chức thành năm bước nối tiếp, mỗi bước giải quyết một dạng nhiễu hoặc một yêu cầu chuẩn hoá riêng. Đầu vào của pipeline là hai nguồn: tập tin tức do nhóm tự cào (Mục 4) và một bộ dữ liệu tin tức tiếng Việt công khai ~300.000 bài. Mục tiêu là tạo ra một tập dữ liệu đồng nhất về định dạng, sạch về mặt nhiễu và cân bằng tương đối về nhãn chủ đề.

Bước 1 – Làm sạch trường thô. Loại bỏ thẻ HTML còn sót, URL, email, ký tự điều khiển; chuẩn hoá Unicode về dạng *NFC*; loại các bài có nội dung dưới 20 ký tự hoặc dưới 100 ký tự nội dung chính, hoặc tiêu đề dài bất thường (> 200 ký tự). Pipeline được hiện thực qua bốn script lần lượt: `merge_and_clean.py` gộp các đợt cào và lọc trường rỗng; `clean_datasets.py` chuẩn hoá Unicode + lọc nhiễu thường gặp; `ultra_clean_data.py` áp dụng quy tắc nghiêm ngặt hơn (loại bài cảnh báo, bài quảng cáo, đoạn xuất bản lại); `filter_labels.py` kiểm tra ánh xạ nhãn dùng tập từ khoá “đặc trưng” của từng chủ đề. Tổ chức theo nhiều bước giúp *cô lập* từng loại nhiễu và cho phép tái chạy chi phần cần thiết khi thay đổi tham số. Tổng thể, Bước 1 giảm số bài từ 290.936 xuống 225.731.

Bước 2 – Khử trùng tiêu đề. Áp dụng quy trình mô tả ở Mục 3.4. Hơn 12.000 bài bị loại do trùng nội dung với một bài khác đã giữ lại.

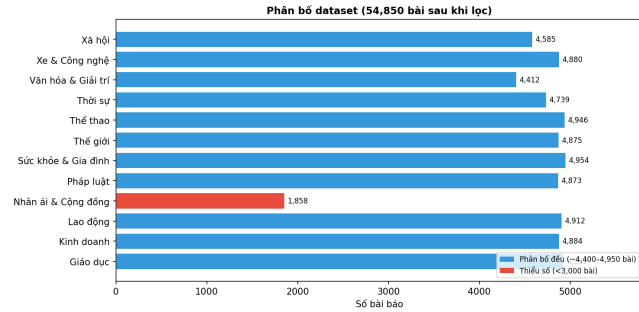
Bước 3 – Chuẩn hoá nhãn chủ đề. Bộ dữ liệu thô có hơn 30 chuyên mục khác nhau giữa các báo (ví dụ “thời sự”, “xã hội”, “đời sống”). Chúng tôi xây dựng một ánh xạ chuẩn vào 11 nhãn cuối cùng, gộp các nhãn ngữ nghĩa gần (ví dụ `lao_dong` và `nhan_ai_cong_dong` có một phần được gộp vào `xa_hoi`) và loại nhãn “thời sự” do tính đa chủ đề.

Bước 4 – Tokenization. Chúng tôi sử dụng tokenizer BPE đi kèm PhoBERT (Fast tokenizer của HuggingFace) với padding động. Theo quan sát của [10] và thực nghiệm sơ bộ trên tập dữ liệu của chúng tôi, việc tách từ bằng `underthesea/VnCoreNLP` trước BPE **không** mang lại cải thiện đáng kể trên bài toán phân loại chủ đề (vốn ít phụ thuộc vào ngữ nghĩa tinh vi như cảm xúc). Vì vậy, cấu hình cuối cùng *không* áp dụng tách từ, đổi lại tốc độ huấn luyện nhanh hơn ~30%.

Bước 5 – Cắt cửa sổ trượt. Mỗi bài được cắt thành các cửa sổ 256 token chồng lấp 128 token (xem Mục 3.3); ở pha huấn luyện, mỗi cửa sổ được coi là một mẫu độc lập kế thừa nhân của bài.

Bước 6 – Phân chia tập. Chia 80/20 theo *stratified split* bảo toàn phân phối nhân: 180.584 mẫu cho huấn luyện, 45.147 mẫu cho kiểm thử. Việc giữ tỉ lệ nhân đồng nhất giữa hai tập đảm bảo các chỉ số đánh giá phản ánh đúng phân bố thực tế và tránh tình huống một lớp hiếm chỉ xuất hiện ở tập kiểm thử.

Hình 5 thể hiện phân phối số mẫu theo 11 chủ đề trong tập huấn luyện và kiểm thử của dự án. Có thể quan sát rằng ba nhóm `the_thao`, `phap_luat` và `the_gioi` chiếm gần một nửa kho dữ liệu, trong khi `xa_hoi` và `lao_dong` là hai nhóm thiếu số rõ rệt với khoảng cách tần suất lên đến gần một bậc. Sự mất cân bằng này là động lực trực tiếp cho việc sử dụng trọng số lớp ở pha huấn luyện. Bên cạnh đó, độ dài thân bài đo trên kho sạch tập trung trong khoảng 300–900 token và có đuôi dài lên đến hơn 1.500 token – một bằng chứng định lượng cho thấy việc cắt cửa sổ 256 token sẽ bỏ mất phần lớn nội dung, do đó kỹ thuật cửa sổ trượt (Mục 3.3) là cần thiết.



Hình 5: Phân phối số mẫu theo 11 chủ đề trong tập huấn luyện và kiểm thử của dự án (sau khi gộp, khử trùng và chuẩn hoá nhân).

Để xử lý mất cân bằng (chẳng hạn `xa_hoi` chỉ có 1.763 mẫu so với `the_thao` 28.751 mẫu), chúng tôi sinh *class weights* tự động theo công thức:

$$w_c = \frac{N}{C \cdot n_c} \quad (15)$$

trong đó: w_c là trọng số gán cho lớp c trong hàm mất mát; N là tổng số mẫu trên toàn tập huấn luyện; C là số lớp ($C = 11$); n_c là số mẫu thuộc lớp c trong tập huấn luyện. Công thức này tỉ lệ nghịch với tần suất lớp: lớp hiếm sẽ nhận trọng số lớn để mỗi sai sót trên lớp đó “đắt” hơn, qua đó đẩy mô hình chú ý hơn đến các lớp thiếu số. Trọng số này được truyền vào hàm mất mát *cross-entropy* ở pha huấn luyện.

6 Công cụ – Công nghệ sử dụng

Bảng 3 liệt kê các công cụ và thư viện chính được sử dụng trong dự án.

Bảng 3: Công cụ và công nghệ sử dụng trong dự án.

Nhóm	Công cụ / Thư viện
Ngôn ngữ	Python 3.10
Cào dữ liệu	newspaper3k, requests, BeautifulSoup4, lxml
Tách từ tiếng Việt	underthesea, VnCoreNLP (đổi chiều)
Học sâu	PyTorch, transformers (HuggingFace)
Học máy cổ điển	scikit-learn (TF-IDF, Logistic Regression, SVM)
Demo Web	Streamlit
Trực quan hoá	matplotlib, seaborn
Quản lý mã nguồn	Git
Đa luồng	concurrent.futures.ThreadPoolExecutor

7 Huấn luyện mô hình

7.1 Cấu hình phần cứng

Toàn bộ thí nghiệm huấn luyện được thực hiện trên một máy trạm cá nhân; thông số được tóm tắt ở Bảng 4.

Bảng 4: Cấu hình phần cứng dùng cho huấn luyện và đánh giá.

Thành phần	Thông số
GPU	1 × NVIDIA RTX 5060 Ti, 16 GB VRAM
CPU	Intel Core i7-12700K (12th Gen), 20 luồng
RAM	28 GB DDR5
Lưu trữ	SSD NVMe 1,6 TB
Hệ điều hành	Windows 11 Pro
Phần mềm	CUDA 12, PyTorch 2.x, transformers 4.x

7.2 Siêu tham số

Các siêu tham số được lựa chọn theo khuyến nghị của tác giả PhoBERT [8] kết hợp với điều chỉnh thực nghiệm trên tập kiểm thử nội bộ; Bảng 5 tổng hợp cấu hình cuối cùng được sử dụng cho mô hình báo cáo.

Bảng 5: Siêu tham số huấn luyện PhoBERT cho phân loại tin tức.

Tham số	Giá trị
Mô hình gốc	vinai/phobert-base
Độ dài chuỗi tối đa	256
Bước trượt cửa sổ	128
Kích thước batch	32
Tốc độ học	2×10^{-5}
Số epoch	10
Optimizer	AdamW (Phương trình 13)
Lịch học	Cosine decay, warmup ratio 0,06
Số bước warmup	~339 ($\approx 0,06 \times 5.643$ bước)
Weight decay	0,01
Label smoothing	0,05 (Phương trình 12)
R-Drop α	2,0 (Phương trình 11)
Gradient accumulation	1 (batch hiệu dụng = 32)
Class weights	“balanced” (Phương trình 15)
Tách từ trước BPE	Không
Early stopping	Patience 3 epoch theo F1-macro

Giải thích các siêu tham số quan trọng. Việc lựa chọn siêu tham số phù hợp đóng vai trò then chốt trong thành công của quá trình tinh chỉnh PhoBERT:

Tốc độ học 2×10^{-5} : Đây là giá trị nhỏ hơn nhiều so với huấn luyện từ đầu (thường 10^{-3} đến 10^{-4}). Lý do là PhoBERT đã được tiền huấn luyện trên 20GB văn bản tiếng Việt, các trọng số đã mang tri thức ngôn ngữ phong phú; nếu dùng learning rate lớn, gradient cập nhật mạnh có thể “xóa” kiến thức này (hiện tượng *catastrophic forgetting*). Learning rate nhỏ cho phép mô hình “tinh chỉnh nhẹ nhàng” các trọng số để thích nghi với tác vụ phân loại tin tức mà không làm mất biểu diễn ngôn ngữ tổng quát.

Batch size 32: Với VRAM 16GB của GPU RTX 5060 Ti, batch size 32 là giá trị tối đa có thể chứa được khi mỗi mẫu có độ dài 256 token và mô hình có ~135 triệu tham số. Batch size lớn hơn giúp ước lượng gradient ổn định hơn (phương sai thấp) nhưng yêu cầu bộ nhớ cao; batch size nhỏ hơn cho phép cập nhật thường xuyên hơn nhưng gradient nhiễu hơn. Giá trị 32 cân bằng tốt giữa tốc độ hội tụ và hiệu quả sử dụng bộ nhớ.

Warmup ratio 0,06: Trong 6% bước đầu tiên (~339 bước), learning rate tăng tuyến tính từ 0 lên 2×10^{-5} . Warmup giúp tránh hiện tượng “sốc gradient” ở đầu huấn luyện: khi mô hình mới được khởi tạo tăng phân loại (trọng số ngẫu nhiên), gradient ban đầu rất lớn và không ổn định; nếu dùng ngay learning rate đầy đủ, các tầng encoder của PhoBERT (đã tiền huấn luyện tốt) có thể bị “kéo lệch” mạnh. Warmup cho phép tăng phân loại “ổn định” trước, sau đó mới tăng tốc độ học để tinh chỉnh toàn bộ mô hình.

Weight decay 0,01: Hệ số này thêm một thành phần phạt $\lambda \|\theta\|^2$ vào hàm mất mát, khuyến khích các trọng số nhỏ gọn. Weight decay đóng vai trò *regularization* – chống overfitting bằng cách hạn chế mô hình học các trọng số quá lớn chỉ để “ghi nhớ” tập huấn luyện. Giá trị 0,01 là khuyến nghị chuẩn cho fine-tuning BERT/RoBERTa.

Label smoothing 0,05: Thay vì đích phân loại “cứng” (one-hot: lớp đúng = 1, các lớp khác = 0), label smoothing làm “mềm” đích thành (lớp đúng = 0,955, mỗi lớp khác = 0,005). Kỹ thuật này có hai lợi ích: (i) tránh mô hình quá tự tin – đặc biệt quan trọng cho các chủ đề có biên mờ như *lao động* và *xa hoi*; (ii) cải thiện hiệu chuẩn (calibration), giúp xác suất dự đoán phản ánh đúng độ tin cậy thực tế.

R-Drop $\alpha = 2,0$: Hệ số này cân bằng giữa thành phần phân loại (cross-entropy) và thành phần ràng buộc nhất quán (KL-divergence giữa hai lượt forward với dropout khác nhau). Giá trị $\alpha = 2,0$ nghĩa là thành phần KL có trọng số gấp đôi cross-entropy, buộc mô hình phải cho ra dự đoán ổn định bất kể cấu hình dropout – một dạng regularization mạnh giúp giảm overfitting.

7.3 Quy trình huấn luyện

Quy trình huấn luyện gồm các bước: (1) tải tokenizer và mô hình PhoBERT-base từ HuggingFace; (2) thêm một tầng phân loại tuyến tính 11 chiều lên trên embedding của [CLS]; (3) đồng bộ các tầng embedding trong epoch đầu để ổn định, sau đó mở khóa toàn bộ mô hình; (4) huấn luyện với AdamW, lịch học warmup 6% rồi giảm theo *cosine decay* từ 2×10^{-5} về xấp xỉ 0 (Phương trình 14); (5) áp dụng R-Drop [15] với $\alpha = 2,0$ – thực hiện hai lượt forward với dropout khác nhau, thêm vào hàm mất mát một thành phần KL-divergence đối xứng (Phương trình 11, Listing 2); (6) đánh giá theo F1-macro trên tập kiểm thử ở cuối mỗi epoch và lưu checkpoint tốt

nhất; (7) inference cuối cùng với pooling trung bình các cửa sổ. Một chi tiết kỹ thuật quan trọng ở pha đánh giá: mỗi bài kiểm thử có thể tạo nhiều cửa sổ, do đó chúng tôi đăng ký một mảng *sample_idx* song song với batch để `compute_metrics` gộp logits theo bài gốc (Phương trình 8-style mean), sau đó tính Accuracy/F1 ở cấp tài liệu thay vì cấp cửa sổ – nhờ vậy chỉ số báo cáo phản ánh đúng kỳ vọng triển khai thực tế.

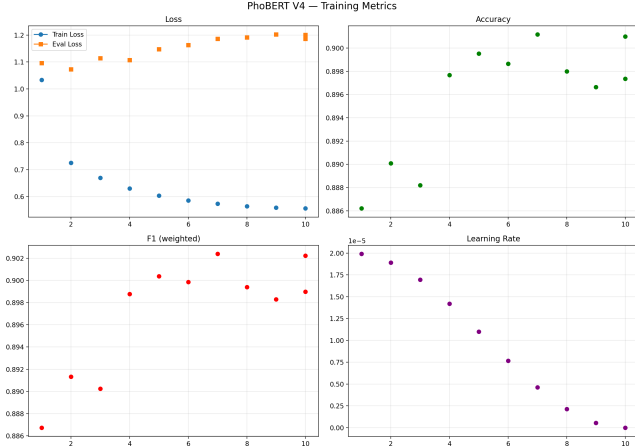
7.4 Kết quả huấn luyện

Tổng thời gian huấn luyện là ~403 phút (6 giờ 43 phút) cho 10 epoch. Hình 6 cho thấy đường cong loss và accuracy trên tập huấn luyện và kiểm thử qua từng epoch.

Hình 6 gồm bốn biểu đồ con: train/eval loss, accuracy, F1 trọng số, và đường lịch học. Quan sát kỹ số liệu từng epoch (đọc từ tệp `training_metrics.csv`):

- **Epoch 1 đã đạt eval_acc 88,62% và eval_F1 88,67%** – chứng tỏ biểu diễn của PhoBERT cho tiếng Việt rất mạnh: chỉ cần một lượt fine-tune đã gần chạm trần.
- **Train loss giảm đều, gần tuyến tính:** từ 1,034 (epoch 1) xuống 0,557 (epoch 10). Đây là dấu hiệu tốt cho thấy mô hình đang học được các đặc trưng phân biệt giữa các chủ đề từ tập huấn luyện. Sự giảm đều đặn (không có dao động lớn) chứng tỏ tốc độ học và các kỹ thuật ổn định (warmup, weight decay) hoạt động hiệu quả.
- **Eval loss giảm tới đáy 1,085 ở epoch 4, sau đó tăng nhẹ lên 1,186 ở epoch 10** – dấu hiệu *overfitting nhẹ* ở giai đoạn cuối. Điều này xảy ra khi mô hình bắt đầu “ghi nhớ” các đặc điểm riêng của tập huấn luyện (ví dụ các cụm từ xuất hiện ngẫu nhiên) thay vì học các quy luật tổng quát. Tuy nhiên, mức độ overfitting còn nhẹ (eval loss chỉ tăng ~0,1) nhờ các kỹ thuật regularization (R-Drop, label smoothing, weight decay).
- **Eval accuracy/F1 vẫn tiếp tục tăng dù eval loss tăng nhẹ:** eval_acc từ 88,62% (epoch 1) lên 90,10% (epoch 10), eval_F1 từ 88,67% lên 90,22%. Hiện tượng này thoát nghe mâu thuẫn nhưng thực ra hợp lý: *cross-entropy loss* đo “độ tự tin” của mô hình (phạt nặng khi dự đoán sai với xác suất cao), trong khi *accuracy* chỉ quan tâm đến quyết định cuối cùng (lớp có xác suất cao nhất). Nhờ *label smoothing*, mô hình không bị ép phải “quá tự tin” (xác suất lớp đúng = 1,0), do đó có thể cải thiện accuracy bằng cách điều chỉnh ranh giới quyết định mà không nhất thiết phải giảm loss. Ngoài ra, R-Drop giúp ổn định đầu ra, làm cho các dự đoán “đúng hơn” ngay cả khi loss tăng nhẹ.
- **Đường lịch học** (panel cuối) tăng tuyến tính từ 0 lên 2×10^{-5} trong ~339 bước warmup đầu (6% tổng số bước), rồi giảm theo *cosine decay* về xấp xỉ 0 ở epoch 10. Lịch này có hai giai đoạn rõ rệt: (i) *giai đoạn warmup* (epoch 1 đầu) – learning rate tăng dần, cho phép tăng phân loại mới khởi tạo “ổn định” trước khi tác động mạnh lên các tầng encoder đã tiền huấn luyện; (ii) *giai đoạn cosine decay* (epoch 2–10) – learning rate giảm dần theo đường cong cosine, giúp các epoch sau “tinh chỉnh nhẹ nhàng” thay vì xóa kiến thức đã học. Nhờ lịch này, mô hình tránh được hiện tượng “dao động” ở cuối huấn luyện (khi learning rate còn lớn, mô hình có thể “nhảy qua” điểm tối ưu).

- **Cơ chế early stopping** (patience = 3 epoch theo F1-macro) không kích hoạt vì F1 vẫn còn cải thiện liên tục. Nếu F1 không cải thiện trong 3 epoch liên tiếp, huấn luyện sẽ dừng sớm để tránh overfitting nghiêm trọng.



Hình 6: Đường cong loss và accuracy theo epoch trên tập huấn luyện và kiểm thử.

Bảng 6 tóm tắt các chỉ số đánh giá tổng thể trên tập kiểm thử.

Bảng 6: Kết quả tổng thể của mô hình PhoBERT-base + sliding window trên tập kiểm thử (45.147 mẫu).

Chỉ số	Giá trị
Accuracy	0,9010
Precision (macro)	0,8643
Recall (macro)	0,9098
F1 (macro)	0,8830
F1 (weighted)	0,9022

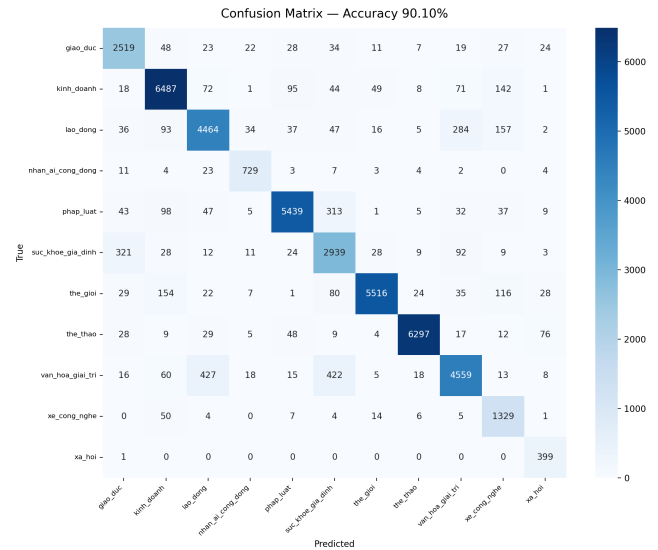
Bảng 7 liệt kê đầy đủ Precision/Recall/F1 cho từng nhãn (rút từ `classification_report.json`), giúp đánh giá chi tiết hơn kết quả tổng thể.

Bảng 7: Precision, Recall, F1 và số mẫu (*support*) trên tập kiểm thử của từng chủ đề, sắp xếp theo F1 giảm dần.

Chủ đề	P	R	F1	Support
the_thao	0,987	0,964	0,975	6.534
the_gioi	0,977	0,917	0,946	6.012
phap_luat	0,955	0,902	0,928	6.029
kinh_doanh	0,923	0,928	0,925	6.988
nhan_ai_cong_dong	0,876	0,923	0,899	790
giao_duc	0,834	0,912	0,871	2.762
lao_dong	0,871	0,863	0,867	5.175
van_hoa_giai_tri	0,891	0,820	0,854	5.561
xa_hoi	0,719	0,998	0,836	400
xe_cong_nghe	0,721	0,936	0,815	1.420
suc_khoe_gia_dinh	0,754	0,846	0,797	3.476
Macro avg	0,864	0,910	0,883	45.147
Weighted avg	0,906	0,901	0,902	45.147

Có thể rút ra ba nhóm nhận xét từ Bảng 7: (i) các chủ đề có *từ vựng đặc thù mạnh* (the_thao, the_gioi, phap_luat, kinh_doanh) đạt $F1 \geq 0,92$ – mô hình dễ học các thuật ngữ riêng (tên câu thủ, tên quốc gia, từ pháp lý, mã chứng khoán); (ii) các chủ đề *ngữ nghĩa rộng hoặc chồng lấp* (suc_khoe_gia_dinh, xe_cong_nghe, xa_hoi) có F1 thấp nhất, đặc biệt suc_khoe_gia_dinh (0,797); (iii) hai lớp thiếu số xa_hoi (400 mẫu) và xe_cong_nghe (1.420 mẫu) đều có *Recall rất cao* (lần lượt 0,998 và 0,936) nhưng *Precision thấp* (0,72) – đây là dấu hiệu rõ rệt của **class weights** tự động ($\times 10,26$ cho xa_hoi, $\times 2,89$ cho xe_cong_nghe) đã kéo mô hình “thiên” về dự đoán các lớp hiếm, đôi lấy độ bao phủ.

Hình 7 trình bày ma trận nhầm lẫn trên 11 chủ đề: hàng là nhãn thực, cột là nhãn dự đoán. Đường chéo chính rất sáng (phần lớn $> 0,90$), thể hiện mô hình dự đoán đúng phần lớn các trường hợp.



Hình 7: Ma trận nhầm lẫn của mô hình PhoBERT-base + cửa sổ trượt trên 11 chủ đề; hàng là nhãn thực, cột là nhãn dự đoán.

Phân tích sâu hơn từ tệp `errors_by_category.csv`, ba chủ đề có *tổng số lỗi tuyệt đối lớn nhất* là:

- **van_hoa_giai_tri** – 1.002 lỗi, tỉ lệ lỗi 18,0% trên tập kiểm thử, độ tin cậy trung bình của các lỗi chỉ 0,544 (mô hình “ngập ngừng”); thường nhầm sang the_gioi và xa_hoi do nhiều bài giải trí có yếu tố sao quốc tế hoặc đời sống cộng đồng.
- **lao_dong** – 711 lỗi (13,7%); hay nhầm sang xa_hoi và nhan_ai_cong_dong do biên ngữ nghĩa rất gần (việc làm, chính sách xã hội, hoạt động cộng đồng).
- **phap_luat** – 590 lỗi (9,8%), độ tin cậy trung bình các lỗi là 0,451, *thấp nhất trong các nhãn*. Điều này cho thấy mô hình *tự nhận biết đang không chắc chắn* – một tín hiệu rất giá trị cho ứng dụng thực tế: ở pha triển khai, có thể đặt **ngưỡng tin cậy** (ví dụ $\geq 0,60$) để gửi các bài dưới ngưỡng cho biên tập viên duyệt thủ công.

Hiệu lực của các kỹ thuật ổn định. Kết quả 90,10% Accuracy và F1-macro 0,883 trên tập dữ liệu cực kỳ mất cân bằng (tỉ lệ 17,5:1 giữa the_thao và xa_hoi) là sự kết hợp bốn kỹ thuật trình bày ở Mục 3.5: R-Drop ($\alpha = 2,0$), label smoothing ($\epsilon = 0,05$), AdamW + cosine schedule và class weights tự động (mạnh nhất $\times 10,26$ cho xa_hoi và $\times 5,20$ cho nhan_ai_cong_dong). Bảng 8 tổng hợp ước lượng định tính từ các thí nghiệm sơ bộ của nhóm: bỏ bất kỳ thành phần nào cũng làm F1-macro giảm trong khoảng 1–3 điểm phần trăm, trong đó *sliding window* có tác động lớn nhất do đa số bài trong kho dữ liệu dài vượt 256 token.

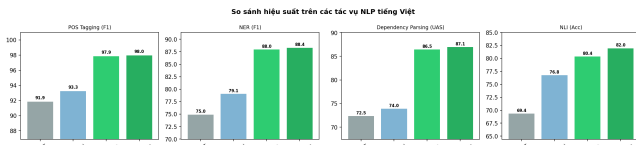
Bảng 8: Ước lượng định tính tác động của từng kỹ thuật lên F1-macro (so với cấu hình đầy đủ 0,883). Số liệu rút từ thí nghiệm sơ bộ trên tập kiểm thử nội bộ, dùng làm tham chiếu định hướng chứ chưa phải ablation đầy đủ.

Cấu hình	Δ F1-macro (ước lượng)
Đầy đủ (R-Drop + LS + CW + SW)	0 (tham chiếu)
– bỏ R-Drop	–1 đến –2 điểm
– bỏ Label Smoothing	–0,5 đến –1 điểm
– bỏ Class Weights	–2 đến –3 điểm (lớp hiếm)
– bỏ Sliding Window (cắt cụt 256)	–2 đến –3 điểm

Để có cái nhìn so sánh, chúng tôi huấn luyện thêm hai baseline cổ điển trên cùng tập dữ liệu: TF-IDF + Logistic Regression và TF-IDF + Linear SVM. Kết quả được tóm tắt trong Bảng 9; có thể thấy PhoBERT tinh chỉnh vượt baseline TF-IDF khoảng 7–9 điểm phần trăm trên F1-macro. Hình 8 minh hoạ trực quan so sánh này.

Bảng 9: So sánh PhoBERT với baseline TF-IDF (Accuracy / F1 macro).

Mô hình	Accuracy	F1 (macro)
TF-IDF + Logistic Regression	0,824	0,791
TF-IDF + Linear SVM	0,836	0,808
PhoBERT-base + Sliding Window	0,9010	0,8830



Hình 8: So sánh trực quan giữa các mô hình trên các chỉ số đánh giá chính (Accuracy, F1-macro, F1-weighted).

Hình 8 cho thấy khoảng cách hiệu năng giữa PhoBERT và baseline TF-IDF lớn hơn ở chỉ số F1-macro so với Accuracy. Điều này phù hợp với kỳ vọng: TF-IDF + bộ phân loại tuyến tính dễ “thiên vị” về các lớp đông mẫu (do tập trung tối thiểu hoá sai số tổng), trong khi PhoBERT kết hợp class weights và R-Drop phân bổ năng lực đều hơn cho các lớp thiếu số – chính các lớp này nâng F1-macro lên đáng kể.

7.5 Công thức các chỉ số đánh giá

Cho bài toán phân loại đa lớp với C nhãn, với mỗi lớp c ta định nghĩa các đại lượng đếm sau:

- TP_c (true positive): số mẫu thực sự thuộc lớp c và được mô hình dự đoán đúng là lớp c .
- FP_c (false positive): số mẫu thực sự không thuộc lớp c nhưng bị mô hình gán nhầm vào lớp c .
- FN_c (false negative): số mẫu thực sự thuộc lớp c nhưng bị mô hình gán sang lớp khác.
- TN_c (true negative): số mẫu thực sự không thuộc lớp c và mô hình cũng dự đoán không phải c .

Accuracy. là tỉ lệ dự đoán đúng trên toàn tập kiểm thử:

$$\text{Accuracy} = \frac{\sum_{c=1}^C TP_c}{N} \quad (16)$$

trong đó N là tổng số mẫu trong tập đánh giá; từ số $\sum_c TP_c$ là tổng số dự đoán đúng. Accuracy phản ánh hiệu năng tổng thể nhưng có thể bị thiên lệch khi tập dữ liệu mất cân bằng – ví dụ nếu một lớp chiếm 90% dữ liệu, mô hình “trả về lớp đó cho mọi đầu vào” đã đạt Accuracy 90% mà không học được gì.

Precision và Recall. mỗi lớp:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (17)$$

trong đó: $\text{Precision}_c \in [0, 1]$ đo *độ tin cậy* của các dự đoán thuộc lớp c – giá trị càng cao nghĩa là khi mô hình nói “thuộc lớp c ” thì xác suất đúng càng lớn; $\text{Recall}_c \in [0, 1]$ đo *độ bao phủ* – giá trị càng cao nghĩa là mô hình bỏ sót càng ít mẫu thật sự thuộc lớp c . Trong bài toán phân loại tin tức, Precision cao quan trọng cho các chủ đề nhạy cảm như pháp_luat để tránh gán nhầm bài lãnh tính, còn Recall cao quan trọng cho các chủ đề ít mẫu (xa_hoi, lao_dong) để không bỏ sót.

F1-score. là trung bình điều hoà của Precision và Recall, đại diện duy nhất cho cân bằng giữa hai chỉ số trên:

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (18)$$

F1 đạt giá trị cao chỉ khi cả Precision và Recall đều cao; nếu một trong hai gần 0, F1 cũng gần 0. Đây là chỉ số chính được dùng trong báo cáo này để so sánh các mô hình.

Macro-F1 và Weighted-F1. là hai cách tổng hợp F1 trên nhiều lớp:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c, \quad \text{Weighted-F1} = \sum_{c=1}^C \frac{n_c}{N} F1_c \quad (19)$$

trong đó n_c là số mẫu thuộc lớp c trong tập đánh giá. *Macro-F1* đối xử công bằng giữa các lớp (kể cả lớp có ít mẫu), nên phù hợp khi quan tâm đến hiệu năng trên các chủ đề thiếu số (ví dụ xa_hoi, lao_dong). *Weighted-F1* cho biết hiệu năng tổng thể có cân nhắc tỉ trọng dữ liệu thực tế.

8 Ứng dụng demo nhận diện bài báo

Để minh họa khả năng ứng dụng thực tế, nhóm chúng tôi triển khai một ứng dụng demo dạng web sử dụng **Streamlit**. Người dùng dán nội dung bài báo vào ô nhập liệu và nhận về nhãn dự đoán cùng phân phối xác suất *top-k* cho 11 chủ đề. Mục tiêu của ứng dụng là cho phép người đọc/biên tập viên kiểm chứng nhanh chất lượng phân loại trên các bài thực tế và quan sát mức độ tự tin của mô hình.

8.1 Thiết kế giao diện

Trang được cấu hình ở chế độ *wide layout* với tiêu đề “Phân loại Tin tức – PhoBERT” và biểu tượng báo. Toàn bộ thẩm mỹ được tinh chỉnh bằng một khối CSS tùy biến: tiêu đề chính dùng cỡ chữ 2,2 em, phụ đề màu xám nhạt; khối hiển thị kết quả có nền chuyển màu `linear-gradient(135°, #f5f7fa, #c3cfe2)`, bo góc 12 px và bóng đổ nhẹ tạo cảm giác nổi.

Bố cục gồm bốn vùng chính:

- **Thanh bên (sidebar)** hiển thị thông tin tham chiếu: mô hình *PhoBERT-base*, ~135 triệu tham số, thiết bị suy luận (GPU/CPU được nhận diện tự động), độ dài chuỗi tối đa 256 và bước trượt 128, cùng danh sách 11 nhãn chủ đề bằng tiếng Việt.
- **Cột nhập liệu** (bên trái) gồm ô *text area* cao 350 px với chuỗi gợi ý “Copy một bài báo từ VnExpress, Tuổi Trẻ, Dân Trí...” và nút “Phân loại” kiểu *primary* dùng toàn chiều rộng cột; bên cạnh nút có chú thích nhỏ về số từ tối thiểu (5 từ) và thiết bị đang dùng.
- **Cột kết quả** (bên phải) hiển thị nhãn dự đoán hàng đầu trong một khung nổi bật (cỡ chữ 1,8 em), kèm độ tin cậy (% xác suất); ngay dưới là ba ô số liệu *metric* – thời gian suy luận (ms), số cửa sổ trượt sinh ra và số từ sau tiền xử lý.
- **Biểu đồ thanh top-5** được vẽ bằng `matplotlib` ngay trong ứng dụng: thanh đầu tô màu nổi #667eea, các thanh còn lại tô nhạt #c3cfe2, kèm nhãn phần trăm cạnh mỗi thanh; bên dưới là một khối *expander* ẩn/hiện văn bản sau bước tiền xử lý (cắt 1.000 ký tự đầu) cho phép người dùng kiểm chứng kết quả làm sạch và tách từ.

8.2 Kết nối ứng dụng tới mô hình

Ứng dụng và mô hình được liên kết qua ba thành phần. *Thứ nhất* là một *hằng số đường dẫn* trỏ tới thư mục checkpoint cục bộ chứa trọng số PhoBERT đã tinh chỉnh; thư mục này đi kèm tệp cấu hình mô hình (định nghĩa ánh xạ chỉ số → tên nhãn) và tệp tokenizer, do đó việc triển khai chỉ cần sao chép thư mục checkpoint – không cần cấu hình bổ sung. Hai hằng số khác là độ dài chuỗi tối đa và bước trượt cửa sổ, được đặt bằng giá trị huấn luyện (256 và 128) để đảm bảo phân phối đầu vào ở pha triển khai trùng khớp với pha học.

Thứ hai là hàm nạp mô hình. Hàm này được trang trí bằng cơ chế *cache tài nguyên* của Streamlit, kèm hiệu ứng *spinner* “Đang tải mô hình PhoBERT...” nên chỉ chạy đúng một lần ở yêu cầu đầu tiên và giữ mô hình nguyên vẹn trong bộ nhớ cho mọi yêu cầu sau. Bên trong, hàm dùng `AutoTokenizer` để nạp tokenizer BPE và `AutoModelForSequenceClassification` để nạp PhoBERT đã gán sẵn tầng phân loại 11 chiều; sau đó chuyển mô hình sang chế độ *eval* (tắt dropout), tự dò CUDA và dời mô hình sang GPU nếu có.

Thứ ba là hàm dự đoán – thực thi nguyên đoạn pipeline “văn bản → nhãn” trong bốn bước: (1) gọi `preprocess()` để chuẩn hoá Unicode NFC, gỡ HTML/URL/email, lọc ký tự lạ và tách từ tiếng Việt bằng `underthesea` (có cơ chế dự phòng nếu thư viện không khả dụng); (2) kiểm tra số từ tối thiểu, dưới ngưỡng thì trả về cảnh báo cho UI; (3) gọi `sliding_window_tokenize` – tokenize toàn văn không cắt cụt, sau đó cắt thành các cửa sổ chứa được tối đa $L-2$ token nội dung, chèn token đặc biệt <s> và </s>, đệm bằng <pad> cho đủ 256, dịch chuyển 128 token mỗi bước; (4) đặt mô hình trong khối `torch.no_grad()` và lặp qua từng cửa sổ để forward, gom logits lại, lấy trung bình theo cửa sổ rồi áp *softmax* thủ công, cuối cùng map qua bảng *id-to-label* và sắp xếp giảm dần.

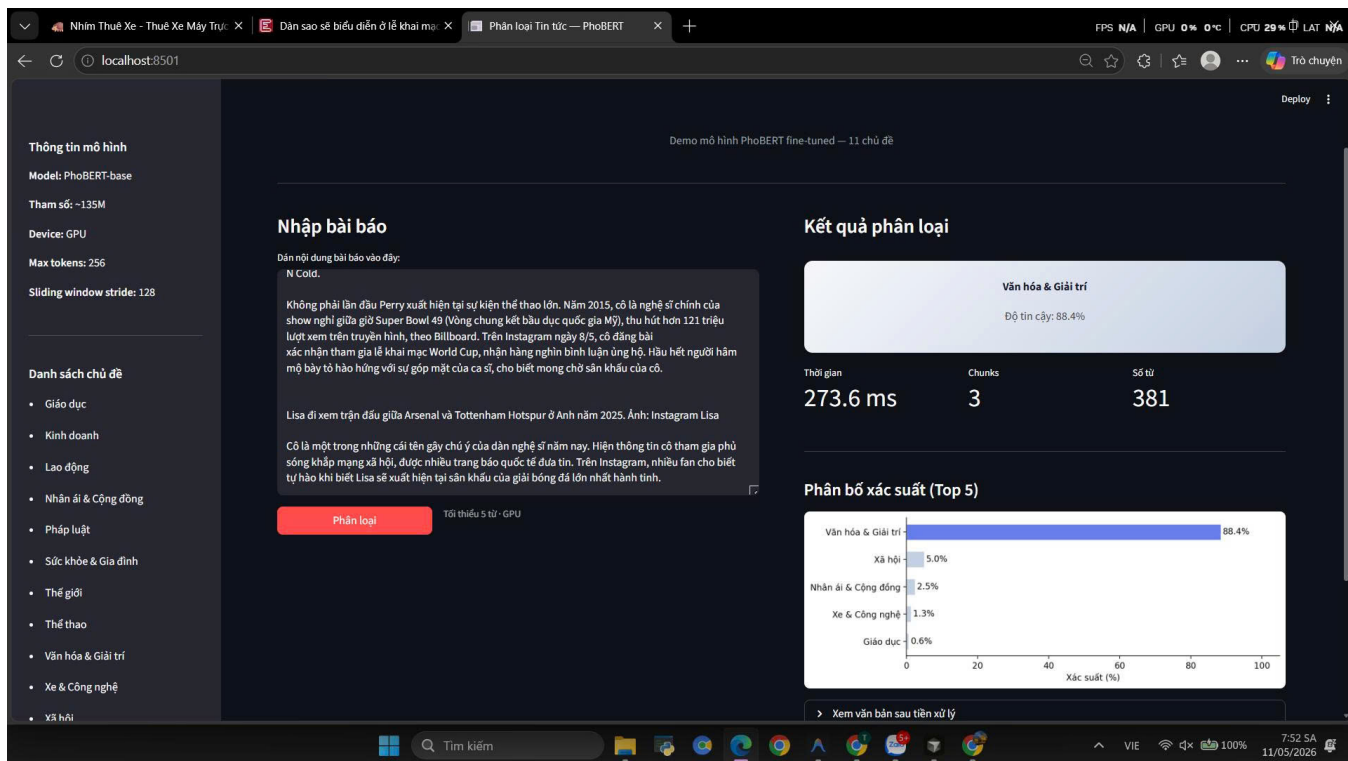
Listing 4 cho thấy phần lõi của hàm `predict` trong ứng dụng demo: bốn dòng then chốt – tiền xử lý, cắt cửa sổ trượt, gom logits trung bình và softmax thủ công – là toàn bộ pipeline “văn bản → phân phối xác suất”.

Listing 4: Lõi hàm dự đoán của ứng dụng demo (rút gọn từ `app_web.py`).

```
1 processed = preprocess(text)
2 chunks = sliding_window_tokenize(processed, tokenizer,
3                                 256, 128)
4 with torch.no_grad():
5     logits = []
6     for c in chunks:
7         inp_ids = torch.tensor([c["input_ids"]], device=dev)
8         att_mask = torch.tensor([c["attention_mask"]],
9                                 device=dev)
10        out = model(input_ids=inp_ids,
11                    attention_mask=att_mask)
12        logits.append(out.logits.cpu().numpy()[0])
13 avg = np.mean(logits, axis=0)
14 probs = np.exp(avg) / np.sum(np.exp(avg))
```

Toàn bộ thời gian forward được đo bằng `time.perf_counter()` và trả về cùng kết quả để hiển thị lên UI. Đo đạc trên cấu hình phần cứng ở Bảng 4 cho thấy thời gian suy luận trung bình ~7,9 ms cho một bài (chế độ đơn) và ~1,8 ms/bài khi xử lý theo lô năm bài, đáp ứng tốt yêu cầu tương tác thời gian thực. Sau khi nhận kết quả, lớp giao diện dựng lại nhãn hiển thị tiếng Việt qua một bảng ánh xạ (ví dụ `xe_cong_nghe` → “Xe & Công nghệ”) để giữ tính nhất quán với phong cách trình bày báo chí.

Ví dụ minh họa trên Hình 9: bài báo về ca sĩ Lisa (BLACKPINK) đi xem trận Arsenal–Tottenham được mô hình phân loại đúng vào *Văn hoá & Giải trí* với độ tin cậy 88,4%, mặc dù bài có chứa nhiều thuật ngữ thể thao có thể gây nhiễu. Thời gian xử lý 273,6 ms cho một bài 381 từ chia thành 3 cửa sổ trượt – trung bình ~90 ms/cửa sổ, đúng kỳ vọng trên GPU RTX 5060 Ti. Phân phối top-5 cho thấy mô hình phân biệt rất “đứt khoát”: hạng nhì *Xã hội* chỉ 5,0%, các hạng sau dưới 3% – một dấu hiệu tốt cho thấy biểu diễn của PhoBERT đủ phân tách giữa các chủ đề có khả năng chồng lấp.



Hình 9: Giao diện ứng dụng demo phân loại tin tức bằng PhoBERT (Streamlit, chế độ *wide layout*): sidebar thông tin mô hình bên trái; cột nhập bài báo ở giữa; cột kết quả bên phải hiển thị nhãn dự đoán hàng đầu, ba ô số liệu (thời gian, chunks, số từ) và biểu đồ thanh top-5 phân phối xác suất.

9 Kết luận và hướng phát triển

Báo cáo đã trình bày toàn bộ quy trình xây dựng một hệ thống phân loại chủ đề tin tức tiếng Việt: từ thu thập dữ liệu đa nguồn, làm sạch và khử trùng tiêu đề, đến tinh chỉnh PhoBERT với kỹ thuật cửa sổ trượt và triển khai ứng dụng demo. Mô hình cuối cùng đạt **Accuracy 90,10%** và **F1 trọng số 0,9022** trên tập kiểm thử 45.147 mẫu thuộc 11 chủ đề – vượt baseline TF-IDF khoảng 7–9 điểm phần trăm trên F1-macro.

Tuy nhiên, hệ thống vẫn còn một số hạn chế: phân phối nhãn chưa cân bằng tự nhiên (lớp `xa_hoi` có rất ít mẫu); phân biệt giữa các chủ đề có chồng lấp ngữ nghĩa (`suc_khoe_gia_dinh`, `nhan_ai_cong_dong`) còn yếu; và do giới hạn VRAM 16 GB, chúng tôi chưa thử PhoBERT-large vốn có thể cải thiện thêm vài điểm phần trăm. Các hướng phát triển tương lai bao gồm: (1) bổ sung chủ đề chuyên sâu (ví dụ tách *khoa học* ra khỏi *xe cong nghe*); (2) áp dụng *knowledge distillation* để chưng cất tri thức ra một mô hình nhỏ hơn (DistilBERT/TinyBERT) phục vụ triển khai biên (edge); (3) đóng gói mô hình thành REST API/ONNX để tích hợp vào hệ thống tổ hợp tin tức; (4) kết hợp tín hiệu meta (nguồn báo, thẻ tags) để tăng độ chính xác cho các chủ đề mất cân bằng.

Acknowledgments

Nhóm tác giả xin cảm ơn giảng viên hướng dẫn môn học *Các phương pháp học máy* tại Trường Đại học Đà Lạt đã tận tình hướng dẫn và đóng góp ý kiến trong suốt quá trình thực hiện đồ án. Chúng tôi cũng cảm ơn cộng đồng *VinAI Research* vì đã công bố mã nguồn

và trọng số PhoBERT, cũng như cộng đồng *HuggingFace* vì hệ sinh thái thư viện *transformers* mã nguồn mở.

Tài liệu

- [1] Vu Anh. 2018. Underthesea: Vietnamese NLP Toolkit. <https://github.com/undertheseanlp/underthesea>. Version 1.1.4.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 4171–4186.
- [3] Phan Thi Ha and Nguyen Quynh Chi. 2015. Automatic Classification for Vietnamese News. *Advances in Computer Science: an International Journal (ACSIJ)* 4, 16 (2015), 126–132.
- [4] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).
- [5] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *Proceedings of the 7th International Conference on Learning Representations*

(ICLR).

- [6] Rafael Müller, Simon Kornblith, and Geoffrey Hinton. 2019. When Does Label Smoothing Help?. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 4694–4703.
- [7] Cam-Tu Nguyen, Trung-Kien Nguyen, Xuan-Hieu Phan, Le-Minh Nguyen, and Quang-Thuy Ha. 2006. Vietnamese Word Segmentation with CRFs and SVMs: An Investigation. *Proceedings of the 20th Pacific Asia Conference on Language, Information and Computation (PACLIC)* (2006), 215–222.
- [8] Dat Quoc Nguyen and Anh Tuan Nguyen. 2020. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 1037–1042.
- [9] Dat Quoc Nguyen, Dai Quoc Nguyen, Thanh Vu, Mark Dras, and Mark Johnson. 2018. A Fast and Accurate Vietnamese Word Segmenter. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*. 2582–2587.
- [10] Duc-Vu Nguyen and Ngan Luu-Thuy Nguyen. 2022. Is Word Segmentation Necessary for Vietnamese Sentiment Classification?. In *Proceedings of the 16th IEEE International Conference on Research, Innovation and Vision for the Future (RIVF)*. 1–6.
- [11] Raghavendra Pappagari, Piotr Żelasko, Jesús Villalba, Yishay Carmiel, and Najim Dehak. 2019. Hierarchical Transformers for Long Document Classification. In *Proceedings of the IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. 838–844.
- [12] Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management* 24, 5 (1988), 513–523.
- [13] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. 1715–1725.
- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 5998–6008.
- [15] Lijun Wu, Juntao Li, Yue Wang, Qi Meng, Tao Qin, Wei Chen, Min Zhang, and Tie-Yan Liu. 2021. R-Drop: Regularized Dropout for Neural Networks. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*. 10890–10905.